Lecture L06

*Blind review — please score this lecture on row **L06** in the Excel form (1–10 per dimension).*

Lecture: Understanding Backpropagation in Neural Networks

The Paradox of Credit Assignment

Imagine you are managing a massive corporation with ten levels of hierarchy. A product fails in the market. As the CEO, you know the final output was wrong, but who is responsible? Is it the salesperson? The factory manager? The raw material procurement officer? Or the strategist five levels deep?

This is the **Credit Assignment Problem**. In a neural network with millions of parameters, we face this exactly: we know the final prediction is wrong, but we don't know which specific weight in which specific hidden layer caused the error. Backpropagation is the mathematical solution to this corporate nightmare—it is a systematic way to distribute "blame" from the output back to the very first weight.

---

1. Prerequisites: The Forward Pass and Computational Graphs

Before we can assign blame, we must trace the path of the "crime." We do this via a computational graph.

The Pipeline of Transformations

A neural network is not a single function, but a composition of many small functions. Data flows from input x to output $\hat{y}$ through a series of layers.

Formal Definition

For any layer l in a Multilayer Perceptron (MLP), the forward pass consists of two operations:

1. **Linear Transformation**: The network computes a weighted sum of the previous layer's outputs and adds a bias:
$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$$
where $W^{(l)}$ represents the weight matrix, $b^{(l)}$ is the bias vector, and $a^{(l-1)}$ is the activation from the previous layer.

2. **Nonlinear Activation**: The sum is passed through an activation function σ (e.g., ReLU or Sigmoid):
$$a^{(l)} = \sigma(z^{(l)})$$

The final output $\hat{y}$ is evaluated by a **Cost Function** $J(\theta)$, which measures the discrepancy between the prediction and the ground truth.

The "Linear Collapse" Tension: Why bother with σ? If we remove the activation functions, we are simply multiplying matrices. Mathematically, $W_2(W_1x) = (W_2W_1)x = W_{combined}x$. No matter how many layers you add, the network remains a simple linear regression. The non-linearity is what allows the network to "bend" the coordinate space to fit complex data.

Try this yourself: Imagine a 2D plane of data points where red dots form a circle and blue dots are outside it. Can a single straight line (linear transformation) separate them? No. But if you "warp" the plane (non-linear activation), can you make them separable?

---

2. The Core Mechanism: The Chain Rule

To minimize the cost J, we need the gradient $\frac{\partial J}{\partial w}$. This tells us: *"If I increase this weight by a tiny amount, does the error go up or down?"*

The Analogy: The Gear Train

Think of a neural network as a series of connected gears. If you turn the final gear (the Loss), it turns the previous gear (the Activation), which turns the one before it (the Sum), which finally turns the input weight. The "gradient" is the gear ratio. If the final gear has a ratio of 2:1 and the second has 3:1, the total effect on the first gear is $2 \times 3 = 6$.

Formalism: The Chain Rule

If z depends on y, and y depends on x, the rate of change is the product of the intermediate rates:

$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y} \cdot \frac{\partial y}{\partial x}$$

Worked Example: Gradient of a Single Neuron

Consider the path: Input $(x) \xrightarrow{w}$ Sum $(z) \xrightarrow{\sigma}$ Activation $(a) \xrightarrow{L}$ Loss (J).

To find $\frac{\partial J}{\partial w}$, we decompose it into local derivatives: 1. **The Output Error**: $\frac{\partial J}{\partial a}$ (How much the loss changes with the activation). 2. **The Activation Slope**: $\frac{\partial a}{\partial z} = \sigma'(z)$ (How much the activation changes with the sum). 3. **The Input Contribution**: $\frac{\partial z}{\partial w} = x$ (How much the sum changes with the weight).

Combining these:

$$\frac{\partial J}{\partial w} = \frac{\partial J}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}$$

The Tension of the Zero-Gradient: What happens if the activation function σ is a flat line at the current value of z? $\sigma'(z)$ becomes 0. Because we are multiplying, the entire gradient becomes 0. The "gear" has snapped; no matter how much the loss screams for a change, the signal cannot pass through that neuron to the weights behind it.

---

3. Computational Efficiency: Dynamic Programming

A naive implementation of the chain rule would be catastrophic. In a deep network, many weights share the same "upstream" paths to the loss. Calculating them independently would be like re-calculating the entire corporate hierarchy for every single employee.

Memoization: The "Save and Reuse" Strategy

Backpropagation is essentially **Reverse-Mode Automatic Differentiation**. Instead of calculating the gradient for each weight from scratch, we calculate the "error signal" δ for a layer once and reuse it for all weights in that layer.

Analogy: If you know that $10 \times 2 = 20$, and you need to calculate $(10 \times 2) \times 5$ and $(10 \times 2) \times 7$, you don't multiply 10×2 twice. You store "20" in memory and multiply it by 5 and 7 respectively.

Formalism: The Error Signal

We define the error signal for layer l as:

$$\delta^{(l)} = \frac{\partial J}{\partial z^{(l)}}$$

By computing $\delta^{(L)}$ (the output layer) and propagating it backward to $\delta^{(L-1)}, \delta^{(L-2)}$..., we reduce the complexity from exponential to linear relative to the number of layers.

Try this yourself: If you are implementing this in code, why is it necessary to keep the values of $a^{(l)}$ (the activations) from the forward pass in memory? (Hint: Look at the $\frac{\partial z}{\partial w} = x$ term in the chain rule).

---

4. Implementation in MLPs

We use **Minibatch Stochastic Gradient Descent (SGD)** to update weights, balancing the stability of the full dataset with the speed of single samples.

The Backpropagation Algorithm

```
# Pseudocode for one training iteration
1. Forward Pass:
   for l = 1 to L:
       z[l] = W[l] @ a[l-1] + b[l]
       a[l] = sigma(z[l])
   cost = J(a[L], y_true)

2. Output Error:
   # The starting point of the "blame"
   delta[L] = gradient of J with respect to z[L]

3. Backward Pass:
   for l = L down to 1:
       # Use the stored delta to update local weights
       grad_W[l] = delta[l] @ a[l-1].T
       grad_b[l] = sum(delta[l])

       # Pass the blame to the previous layer using the transpose of W
       if l > 1:
           delta[l-1] = (W[l].T @ delta[l]) * sigma_prime(z[l-1])

4. Update:
   W = W - learning_rate * grad_W
   b = b - learning_rate * grad_b
```

---

5. Limitations and Pitfalls

The Multiplication Trap: Vanishing and Exploding Gradients

Because the chain rule is a sequence of multiplications, the gradient is essentially an exponential function of the network depth. - **Vanishing**: If your gradients are 0.1, then $0.1^{10} \approx 0.0000000001$. The early layers

receive almost no signal and stop learning. - **Exploding**: If your gradients are 2.0, then $2^{10} = 1024$. The weights jump wildly, and the model diverges.

The Saturation Problem

The Sigmoid function $\sigma(z) = \frac{1}{1+e^{-z}}$ is a primary culprit. When z is very large (e.g., 10) or very small (e.g., -10), the curve is almost perfectly flat. - **The Result**: $\sigma'(z) \approx 0$. - **The Impact**: A saturated neuron acts as a "circuit breaker," killing the gradient flow to all preceding layers.

Quick Check: Why did the industry shift toward ReLU ($\sigma(z) = \max(0, z)$)? (Answer: For $z > 0$, the slope is exactly 1. Multiplying by 1 repeatedly does not shrink the gradient, effectively solving the vanishing gradient problem for positive activations).

---

6. Generalizations and Modern Context

Backpropagation is the universal engine for almost all deep learning: - **CNNs**: Backpropagation handles weight sharing. The gradient for a single filter is the *sum* of all gradients calculated at every position the filter touched in the image. - **RNNs**: **Backpropagation Through Time (BPTT)** treats an RNN as a very deep MLP where each "layer" is a different time step. - **Automatic Differentiation**: Modern frameworks (PyTorch, TensorFlow) don't make you write the chain rule. They build a **Dynamic Computational Graph** during the forward pass and automatically traverse it in reverse to compute gradients.